

이진분류

답이 **딱 두 개**인 문제.

스팸이냐 아니냐, 불량이냐 아니냐, 합격이냐 불합격이냐.

Q1 이진분류가 뭐야

세상의 모든 질문을
"응" 아니면 "아니"로만 답하게 만든 문제

이진(二進)은 '둘'이라는 뜻입니다. 답이 두 종류뿐인 문제를 컴퓨터에게 맞게 하는 거예요.

문 제	1이 라고 부를 쪽	0이 라고 부를 쪽
이 메일 스팸이야?	스팸	정상
이 부품 불량이야?	불량	정상
이 손님 또 올까?	재방문	안 올
이 환자 병 있어?	양성	음성

컴퓨터는 글자를 모르니까 **한쪽을 1, 한쪽을 0**으로 바꿔서 알려줍니다.

Q2 가장 먼저 정할 것 — 누가 '1'인가

이게 이진분류에서 **제일 중요한 결정**입니다. 그냥 아무거나 1로 정하면 안 돼요.

규칙: 찾고 싶은 것, 놓치면 큰일 나는 것을 1(양성)로 둡니다.

어느 쪽을 1로?

1 · 양성 (병 있음)

놓치면 사람이 위험 → 반드시 찾아야 함
전체의 2% 밖에 없는 소수

0 · 음성 (병 없음)

틀려도 다시 검사하면 됨
전체의 98%인 다수

보통 '적고 중요한 쪽'이 1이 됩니다.

Q3 정확도만 보면 속습니다

환자 1000명 중 진짜 환자가 **20명**뿐이라고 해봅시다. 여기 아무 생각 없는 모델이 하나 있습니다.

"전부 다 정상입니다"

— 아무것도 안 보고 그냥 이렇게만 대답하는 모델

이 모델의 정확도는? $980 \div 1000 = 98\%$ 입니다.

정상 980명 — 전부 맞힘 ✓

환자 20명 — 한 명도 못 찾음 ✗

정확도 98% · 그런데 쓸모는 0

정확도 98%인데 정작 찾아야 할 환자는 한 명도 못 찾았습니다.

한쪽이 압도적으로 많은 데이터를 **불균형 데이터**라고 합니다.

이럴 때 정확도는 **거짓말쟁이**예요. 그래서 다른 점수표가 필요합니다.

Q4 채점표 — 혼동행렬

모델의 답을 네 칸으로 나눠서 봅니다. 이 표를 혼동행렬(confusion matrix)이라고 해요.

	모델이 "환자다"	모델이 "정상이다"
진짜 환자	잘 잡았다 ✓ 환자를 환자라고 함 TP	놓쳤다 ✗ 환자인데 정상이래 ← 병원에선 이게 제일 무섭다 FN
진짜 정상	헛다리 △ 정상인데 환자래 놀라긴 하지만 재검사하면 됨 FP	잘 넘겼다 ✓ 정상을 정상이라고 함 TN

초록 두 칸은 맞힌 것, 빨강·노랑 두 칸은 틀린 것. 틀리는 방식이 두 가지라는 게 핵심입니다.

놓치는 것(FN)과 헛다리 짚는 것(FP)은 완전히 다른 실수입니다. 병원에선 놓치는 게 치명적이고, 스팸함에선 헛다리가 더 짜증나죠.

Q5 정밀도와 재현율

이 두 가지 실수를 각각 얼마나 하는지 재는 점수입니다. 이름은 어렵지만 뜻은 간단해요.

재현율 (Recall) — "진짜 환자 중에 몇 명이나 찾아냈어?"

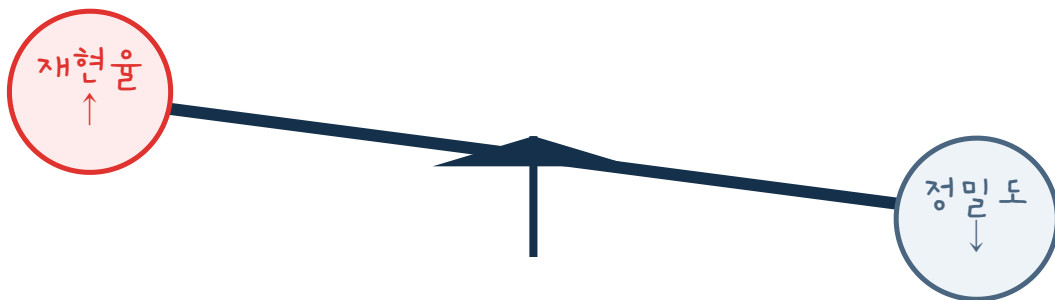
놓치기 싫을 때 보는 점수. 암 검사, 화재 감지, 지진 경보.

정밀도 (Precision) — "환자라고 한 것 중에 진짜가 몇 명이야?"

헛소리 하기 싫을 때 보는 점수. 스팸 필터, 상품 추천.

의심스러우면 다 환자라고 해버리면?

→ 놓치는 사람은 없어지지만(재현율 ↑), 헛다리가 늘어남(정밀도 ↓)



한쪽을 올리면 보통 다른 쪽이 내려갑니다 (시소 관계)

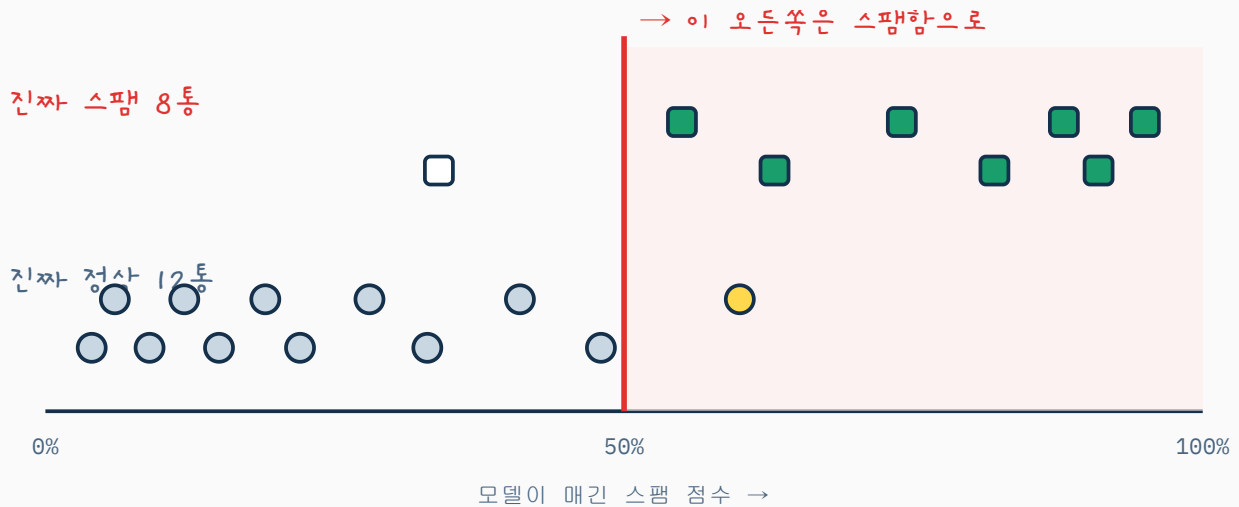
그래서 두 점수를 하나로 합친 게 F1입니다. 둘 다 높아야 F1이 높아지고, 한쪽만 잘해서는 점수가 안 올라가요.

Q6 직접 기준선을 옮겨보기

스팸 판정 실험실

메일 20통. 위쪽은 진짜 스팸 8통, 아래쪽은 정상 12통. 모델이 매긴 '스팸 점수' 위치에 놓여 있습니다.

빨간 기준선을 옮기면 판정 결과가 바뀝니다.



스팸이라고 부를 기준선

50%

잘 잡음 TP	놓침 FN	헛다리 FP	재현율	정밀도	F1
7	1	1	88%	88%	0.88

스팸 1통을 놓쳤고, 정상 메일 1통을 잘못 걸렸습니다.

기준선을 **왼쪽 끝**으로 밀면 놓치는 건 없지만 정상 메일이 대거 스팸함으로 갑니다. **오른쪽 끝**으로 밀면 헛다리는 없지만 스팸이 다 통과해요. **F1이 가장 높은 지점**이 대체로 좋은 타협점입니다.

Q7 ROC-AUC — 기준선을 안 정하고 채점하기

위 실험에서 봤듯이, 점수는 기준선을 어디 두냐에 따라 계속 바뀝니다. 그럼 "이 모델 자체는 얼마나 똑똑한가?"는 어떻게 재조?

기준선을 0%부터 100%까지 전부 시험해보고
그 성적을 통째로 하나의 숫자로 만든 것 = ROC-AUC

AUC 값	뜻
1.0	완벽. 스팸과 정상이 딱 갈림
0.8~0.9	쓸 만함
0.5	동전 던지기와 똑같음 (아무 쓸모 없음)

즉 0.5는 0점이고 1.0이 만점 이라고 생각하면 됩니다. AUC가 0.5면 모델이 아무것도 못 배운 거예요.

Q8 코드로 보기

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, classification_report

# 문제와 정답 나누기 (y는 0 또는 1)
y = train['is_spam']
X = train.drop(columns=['is_spam'])

# stratify=y : 나눌 때도 0과 1의 비율을 그대로 유지하라는 뜻
X_tr, X_val, y_tr, y_val = train_test_split(
    X, y, test_size=0.2, random_state=0, stratify=y)

model = RandomForestClassifier(n_estimators=100, random_state=0)
model.fit(X_tr, y_tr)

pred = model.predict(X_val)          # 0 또는 1
proba = model.predict_proba(X_val)[:, 1]  # 1일 확률 (기준선 실험용)

print('정확도   :', accuracy_score(y_val, pred))
print('f1      :', f1_score(y_val, pred))      # 이진분류는 average 안 써도 됨
print('ROC-AUC :', roc_auc_score(y_val, proba)) # 확률을 넣어야 함! pred 아님
print(classification_report(y_val, pred))
```

자주 틀리는 곳 두 군데

- ① `roc_auc_score` 에는 **0/1이 아니라 확률**을 넣어야 합니다.
- ② `stratify=y` 를 빼먹으면, 데이터를 나눌 때 운 나쁘게 한쪽에 스팸이 몰릴 수 있습니다.

Q9 한 장 정리

단 계	할 일
1. 문제정의	무엇을 1(양성)로 돌지 정한다 — 여기가 전부의 시작
2. 불균형 확인	<code>y.value_counts()</code> 로 0과 1의 개수를 본다
3. 분할	<code>stratify=y</code> 로 비율 유지하며 나눈다
4. 채점	불균형이면 정확도 말고 F1·재현율·ROC-AUC
5. 기준선 조정	모델을 다시 학습하지 않고도 성능을 바꿀 수 있다

이것만 기억하면 됩니다

- ① 정확도는 불균형 데이터에서 거짓말을 한다
- ② 틀리는 방식은 **놓침(FN)**과 **헛다리(FP)** 두 가지고, 상황마다 더 나쁜 쪽이 다르다
- ③ 기준선은 고정값이 아니라 **내가 정하는 것이다**

Q10 Level별 코드 작성하기

옆에 주피터 노트북을 열고, 복사 버튼으로 코드를 붙여넣어 따라 해보세요. 쉬운 것부터 실전(캐글 공공데이터)까지 3단계입니다.

코드 해석 ▼을 누르면 한 줄씩 쉽게 풀어 줍니다.

LEVEL 1 단순 — 내장 유방암 데이터로 악성/양성 맞추기

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X, y = load_breast_cancer(return_X_y=True)    # 0=악성, 1=양성
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0, stratify=y)
model = LogisticRegression(max_iter=5000).fit(X_tr, y_tr)
print('정확도:', round(accuracy_score(y_te, model.predict(X_te)), 3))
```

```
X, y = load_breast_cancer(return_X_y=True)
```

유방암 세포 데이터를 특성 X와 정답 y(0=악성,1=양성)로 바로 받습니다.

```
train_test_split(..., stratify=y)
```

학습용/시험용으로 나눕니다. stratify=y는 양성·악성 비율을 양쪽에 똑같이 유지.

```
LogisticRegression(max_iter=5000).fit(X_tr, y_tr)
```

로지스틱 회귀로 학습. max_iter는 계산 반복 상한(경고 방지).

```
accuracy_score(y_te, model.predict(X_te))
```

시험셋에서 맞힌 비율(정확도)을 봅니다.

LEVEL 2 복잡 — 스케일링 + ROC-AUC + 정밀도/재현율

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, classification_report

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0, stratify=y)
sc = StandardScaler()
X_tr_s = sc.fit_transform(X_tr)      # train만 기준 학습
X_te_s = sc.transform(X_te)         # test는 변환만

model = LogisticRegression(max_iter=5000).fit(X_tr_s, y_tr)
proba = model.predict_proba(X_te_s)[: , 1]
print('ROC-AUC:', round(roc_auc_score(y_te, proba), 3))
print(classification_report(y_te, model.predict(X_te_s)))
```

```
sc.fit_transform(X_tr) / sc.transform(X_te)
```

train에서만 기준을 배우고, test는 그 기준으로 변환만(누수 방지).

```
model.predict_proba(X_te_s)[: , 1]
```

0/1 대신 '양성일 확률'을 꺼냅니다([:,1]=1일 확률).

```
roc_auc_score(y_te, proba)
```

정확도의 함정을 피하는 지표(확률을 넣음). 0.5=찍기, 1=완벽.

```
classification_report(...)
```

클래스별 정밀도·재현율·f1 성적표.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, roc_auc_score

url = 'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
df = pd.read_csv(url)
df['Age'] = df['Age'].fillna(df['Age'].median()) # 나이 결측 채우기
X = pd.get_dummies(df[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']]) # 글자 -> 0/1
y = df['Survived']

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0, stratify=y)
model = RandomForestClassifier(n_estimators=200, random_state=0).fit(X_tr, y_tr)
print('정확도 :', round(accuracy_score(y_te, model.predict(X_te)), 3))
print('ROC-AUC:', round(roc_auc_score(y_te, model.predict_proba(X_te)[:, 1]), 3))
```

```
pd.read_csv(url)
```

캐글 타이타닉을 공개 URL에서 바로 읽습니다(다운로드 불필요).

```
df['Age'].fillna(median)
```

비어 있는 나이를 중앙값으로 채웁니다.

```
pd.get_dummies(...)
```

성별 같은 글자 열을 0/1 열로 바꿉니다(원핫 인코딩).

```
RandomForestClassifier(...).fit
```

랜덤포레스트로 학습 후 정확도·ROC-AUC 확인.